

vLLM专题（十四）-自动前缀缓存

 大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续费

 您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 >

一、介绍

自动前缀缓存（Automatic Prefix Caching，简称 APC）缓存现有查询的 KV 缓存，以便新查询如果与现有查询共享相同的前缀，可以直接重用 KV 缓存，从而跳过共享部分的计算。

注意
有关 vLLM 如何实现 APC 的技术细节，请参阅[此处](#)。

二、在 vLLM 中启用 APC

在 vLLM 引擎中设置 `enable_prefix_caching=True` 以启用 APC。以下是一个示例：

```
import time
from vllm import LLM, SamplingParams

# A prompt containing a large markdown table. The table is randomly generated by GPT-4.
LONG_PROMPT = "You are a helpful assistant in recognizes the content of tables in markdown format. Here is a table as follows.\n# Table\n" + """
| ID | Name | Age | Occupation | Country | Email | Phone Number | Address |
|----|-----|----|-----|-----|-----|-----|-----|
| 1 | John Doe | 29 | Engineer | USA | john.doe@example.com | 555-1234 | 123 Elm St, Springfield, IL |
| 2 | Jane Smith | 34 | Doctor | Canada | jane.smith@example.com | 555-5678 | 456 Oak St, Toronto, ON |
| 3 | Alice Johnson | 27 | Teacher | UK | alice.j@example.com | 555-8765 | 789 Pine St, London, UK |
| 4 | Bob Brown | 45 | Artist | Australia | bob.b@example.com | 555-4321 | 321 Maple St, Sydney, NSW |
| 5 | Carol White | 31 | Scientist | New Zealand | carol.w@example.com | 555-6789 | 654 Birch St, Wellington, NZ |
| 6 | Dave Green | 28 | Lawyer | Ireland | dave.g@example.com | 555-9876 | 987 Cedar St, Dublin, IE |
"""

llm = LLM(model="gpt-4o", enable_prefix_caching=True)
sparams = SamplingParams(temperature=0.0)
outputs = llm.generate(prompt=LONG_PROMPT, sparams=sparams)
```

5-3456		987 Cedar St, Dublin, IE				
7	Emma Black	40	Musician	USA	emma.b@example.com	55
5-1111		246 Ash St, New York, NY				
8	Frank Blue	37	Chef	Canada	frank.b@example.com	55
5-2222		135 Spruce St, Vancouver, BC				
9	Grace Yellow	50	Engineer	UK	grace.y@example.com	55
5-3333		864 Fir St, Manchester, UK				
10	Henry Violet	32	Artist	Australia	henry.v@example.com	55
5-4444		753 Willow St, Melbourne, VIC				
11	Irene Orange	26	Scientist	New Zealand	irene.o@example.com	55
5-5555		912 Poplar St, Auckland, NZ				
12	Jack Indigo	38	Teacher	Ireland	jack.i@example.com	55
5-6666		159 Elm St, Cork, IE				
13	Karen Red	41	Lawyer	USA	karen.r@example.com	55
5-7777		357 Cedar St, Boston, MA				
14	Leo Brown	30	Chef	Canada	leo.b@example.com	55
5-8888		246 Oak St, Calgary, AB				
15	Mia Green	33	Musician	UK	mia.g@example.com	55
5-9999		975 Pine St, Edinburgh, UK				
16	Noah Yellow	29	Doctor	Australia	noah.y@example.com	55
5-0000		864 Birch St, Brisbane, QLD				
17	Olivia Blue	35	Engineer	New Zealand	olivia.b@example.com	55
5-1212		753 Maple St, Hamilton, NZ				
18	Peter Black	42	Artist	Ireland	peter.b@example.com	55
5-3434		912 Fir St, Limerick, IE				
19	Quinn White	28	Scientist	USA	quinn.w@example.com	55
5-5656		159 Willow St, Seattle, WA				
20	Rachel Red	31	Teacher	Canada	rachel.r@example.com	55
5-7878		357 Poplar St, Ottawa, ON				
21	Steve Green	44	Lawyer	UK	steve.g@example.com	55
5-9090		753 Elm St, Birmingham, UK				
22	Tina Blue	36	Musician	Australia	tina.b@example.com	55
5-1213		864 Cedar St, Perth, WA				
23	Umar Black	39	Chef	New Zealand	umar.b@example.com	55
5-3435		975 Spruce St, Christchurch, NZ				
24	Victor Yellow	43	Engineer	Ireland	victor.y@example.com	55
5-5657		246 Willow St, Galway, IE				
25	Wendy Orange	27	Artist	USA	wendy.o@example.com	55
5-7879		135 Elm St, Denver, CO				
26	Xavier Green	34	Scientist	Canada	xavier.g@example.com	55
5-9091		357 Oak St, Montreal, QC				
27	Yara Red	41	Teacher	UK	yara.r@example.com	55
5-1214		975 Pine St, Leeds, UK				
28	Zack Blue	30	Lawyer	Australia	zack.b@example.com	55
5-3436		135 Birch St, Adelaide, SA				
29	Amy White	33	Musician	New Zealand	amy.w@example.com	55
5-5658		159 Maple St, Wellington, NZ				

```

30 | Ben Black | 38 | Chef | Ireland | ben.b@example.com | 55
5-7870 | 246 Fir St, Waterford, IE |
"""

def get_generation_time(llm, sampling_params, prompts):
    # time the generation
    start_time = time.time()
    output = llm.generate(prompts, sampling_params=sampling_params)
    end_time = time.time()
    # print the output and generation time
    print(f"Output: {
        output[0].outputs[0].text}")
    print(f"Generation time: {
        end_time - start_time} seconds.")

# set enable_prefix_caching=True to enable APC
llm = LLM(
    model='lmsys/longchat-13b-16k',
    enable_prefix_caching=True
)

sampling_params = SamplingParams(temperature=0, max_tokens=100)

# Querying the age of John Doe
get_generation_time(
    llm,
    sampling_params,
    LONG_PROMPT + "Question: what is the age of John Doe? Your answer: The age of John Doe is ",
)

# Querying the age of Zack Blue
# This query will be faster since vLLM avoids computing the KV cache of LONG_PROMPT again
get_generation_time(
    llm,
    sampling_params,
    LONG_PROMPT + "Question: what is the age of Zack Blue? Your answer: The age of Zack Blue is ",
)

```

三、示例工作负载

我们描述了两个示例工作负载，APC 可以在这些场景中提供巨大的性能优势：

长文档查询：用户使用不同的查询重复查询同一份长文档（例如软件手册或年度报告）。在这种情况下，APC 允许 vLLM 仅处理一次长文档，而所有未来的请求都可以通过重用其 KV 缓存来避免重新计算此长文档。这使得 vLLM 能够以更高的吞吐量和更低的延迟服务未来的请求。

多轮对话：用户可能会在同一聊天会话中与应用程序进行多次聊天。在这种情况下，APC 允许 vLLM 在所有未来的对话轮次中重用聊天历史的处理结果，从而使 vLLM 能够以更高的吞吐量和更低的延迟服务未来的请求。

四、限制

APC 通常不会降低 vLLM 的性能。尽管如此，APC 仅减少处理查询的时间（预填充阶段），而不会减少生成新 token 的时间（解码阶段）。因此，当 vLLM 大部分时间用于生成查询答案（例如答案较长）时，或者新查询与任何现有查询不共享相同前缀（因此无法重用计算）时，APC 不会带来性能提升。